Coursework Submission: Non-Linear Dynamics, Chaos and Aplications Week 1

Name: Stan Daniels.

Student Number: S1116231

Code:

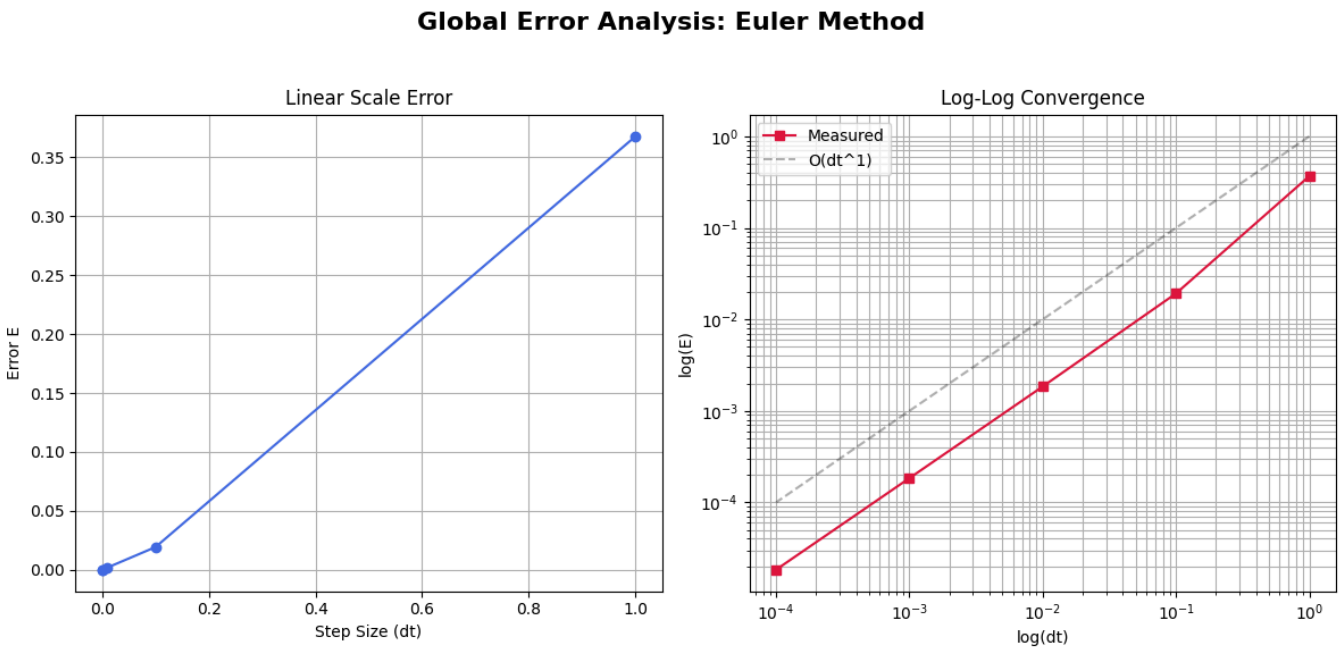
```
# ================================================================
# EXCERSISES WEEK 1 (2.8.3, 2.8.4)
# ================================================================
import numpy as np
import matplotlib.pyplot as plt
import os

class Simulator:
    """
    A numerical simulator for solving Initial Value Problems (IVPs).

    This class provides a framework to estimate solutions using various
numerical
    integration techniques and perform error analysis against exact
solutions.

    Attributes:
        func (callable): The derivative function f(t, x) in dx/dt = f(t,
x).
        x0 (float): Initial value of the state variable at t0.
        t0 (float): Initial time.
        t_end (float): Final time for the simulation.
        method (str): The integration technique ('euler' or
'improved_euler').
    """
    def __init__(self, func, x0, t0, t_end, method='euler'):
        """
        Initializes the ODESimulator with system parameters and solver
choice.

        Args:
            func (callable): Function taking (t, x) and returning dx/dt.
            x0 (float): Starting value of x.
            t0 (float): Starting time.
            t_end (float): Final time to simulate.
            method (str, optional): Numerical method to use. Defaults to
'euler'.
        """
        self.func = func
        self.x0 = x0
```

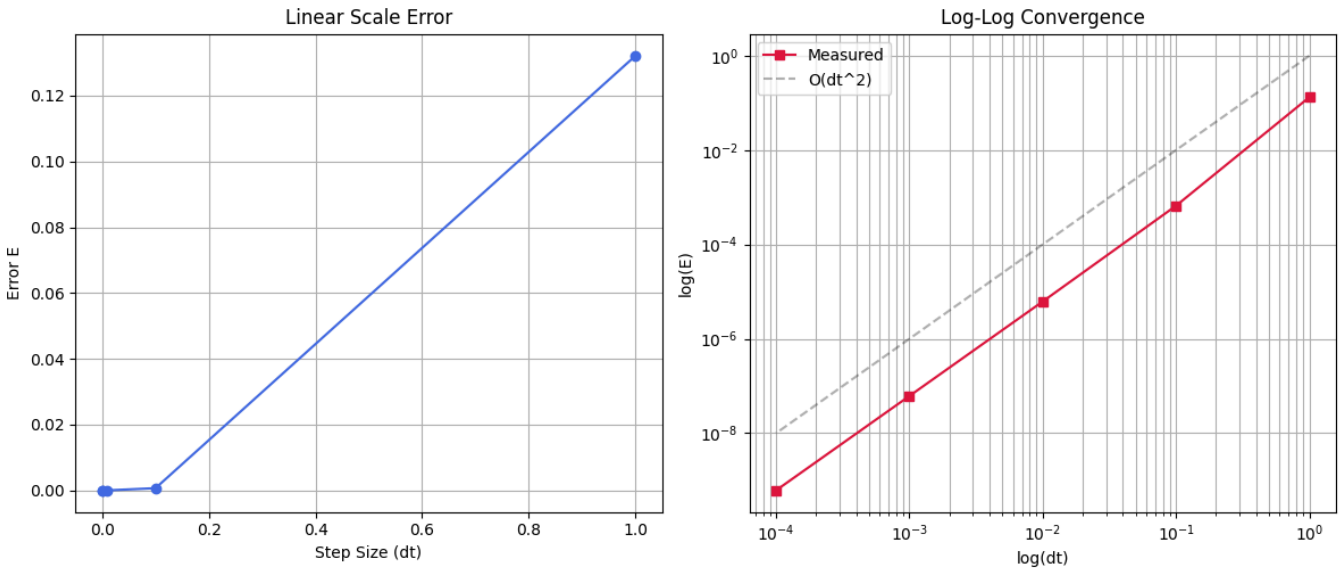
```
        self.t0 = t0
        self.t_end = t_end
        self.method = method.lower()

    def _euler_step(self, x, t, dt):
        """
        Calculates the next state using the Forward Euler Method (1st
Order).

        The formula used is: x_{n+1} = x_n + dt * f(t_n, x_n).

        Args:
            x (float): Current state.
            t (float): Current time.
            dt (float): Step size.

        Returns:
            float: Estimated state at t + dt.
        """
        return x + dt * self.func(t, x)

    def _improved_euler_step(self, x, t, dt):
        """
        Calculates the next state using Heun's Method / Improved Euler (2nd
Order).

        Uses a predictor-corrector approach:
        1. Predict: x_pred = x_n + dt * f(t_n, x_n)
        2. Correct: x_{n+1} = x_n + (dt/2) * [f(t_n, x_n) + f(t_n + dt,
x_pred)]

        Args:
            x (float): Current state.
            t (float): Current time.
            dt (float): Step size.

        Returns:
            float: Estimated state at t + dt.
        """
        k1 = self.func(t, x)
        k2 = self.func(t + dt, x + dt * k1)
        return x + (dt / 2) * (k1 + k2)

    def solve(self, dt):
        """
        Iterates the chosen numerical method from t0 to t_end.

        Args:
            dt (float): The time step size for integration.

        Returns:
            float: The final estimated value of x at t_end.

        Raises:
```

```
            ValueError: If the specified method is not supported.
        """
        t = self.t0
        x = self.x0
        steps = int(round((self.t_end - self.t0) / dt))

        for _ in range(steps):
            if self.method == 'euler':
                x = self._euler_step(x, t, dt)
            elif self.method == 'improved_euler':
                x = self._improved_euler_step(x, t, dt)
            else:
                raise ValueError(f"Unknown method: {self.method}")
            t += dt
        return x

    def generate_error_data(self, n_range):
        """
        Calculates absolute error for a series of decreasing step sizes.

        Assumes the exact solution follows x(t) = exp(-t) for the specific
        test case x' = -x, x(0) = 1.

        Args:
            n_range (range/list): Integers representing step sizes of 10^-
n.

        Returns:
            tuple: (dts, errors) where both are lists of floats.
        """
        dts = [10**-n for n in n_range]
        exact_val = np.exp(-self.t_end) # For x' = -x, x(0)=1
        errors = []

        for dt in dts:
            est = self.solve(dt)
            errors.append(abs(exact_val - est))
        return dts, errors

    def plot_and_save(self, dts, errors):
        """
        Generates linear and log-log plots of error vs. step size.

        Saves the resulting figure as a PNG in a 'plots' subdirectory
        relative to the script location.

        Args:
            dts (list): List of step sizes used.
            errors (list): List of absolute errors corresponding to the
dts.
        """
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 6))

        method_name = "Euler" if self.method == 'euler' else "Improved
```

```
Euler"
        fig.suptitle(f'Global Error Analysis: {method_name} Method',
fontsize=16, fontweight='bold')

        # Plot 1: Linear
        ax1.plot(dts, errors, 'o-', color='royalblue', label=f'Error
({self.method})')
        ax1.set_xlabel('Step Size (dt)')
        ax1.set_ylabel('Error E')
        ax1.set_title(f'Linear Scale Error')
        ax1.grid(True)

        # Plot 2: Log-Log
        ax2.loglog(dts, errors, 's-', color='crimson', label='Measured')
        slope = 1 if self.method == 'euler' else 2
        ax2.loglog(dts, [d**slope for d in dts], 'k--', alpha=0.3,
label=f'O(dt^{slope})')

        ax2.set_xlabel('log(dt)')
        ax2.set_ylabel('log(E)')
        ax2.set_title('Log-Log Convergence')
        ax2.legend()
        ax2.grid(True, which="both")

        plt.tight_layout(rect=[0, 0.03, 1, 0.95])

        base_path = os.path.dirname(os.path.abspath(__file__))
        save_dir = os.path.join(base_path, "plots")

        if not os.path.exists(save_dir):
            os.makedirs(save_dir)

        filename = f"error_analysis_{self.method}.png"
        save_path = os.path.join(save_dir, filename)

        plt.savefig(save_path)
        print(f"Plot saved successfully at: {save_path}")

if __name__ == "__main__":
    def system_dynamics(t, x):
        return -x

    sim_euler = Simulator(system_dynamics, x0=1.0, t0=0.0, t_end=1.0,
method='euler')
    sim_improved_euler = Simulator(system_dynamics, x0=1.0, t0=0.0,
t_end=1.0, method='improved_euler')

    # Run analysis for n = 0, 1, 2, 3, 4
    dts_e, errors_e = sim_euler.generate_error_data(range(5))
    dts_ie, errors_ie = sim_improved_euler.generate_error_data(range(5))

    sim_euler.plot_and_save(dts_e, errors_e)
    sim_improved_euler.plot_and_save(dts_ie, errors_ie)
```

Results:


Global Error Analysis: Euler Method
Linear Scale Error
Error E
Step Size (dt)
Log-Log Convergence
Measured
O(dt^1)
log(E)
log(dt)


Global Error Analysis: Improved Euler Method
Linear Scale Error
Error E
Step Size (dt)
Log-Log Convergence
Measured
O(dt^2)
log(E)
log(dt)